

claude-windows / d4b5afb5-2f03-4b58-8952-573f1381d166

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4b5afb5-2f03-4b58-8952-573f1381d166\subagents\workflows\wf_beb40705-90b\agent-ac9b56c84593a55cc.jsonl`  
- SHA-256: `98189cc35e2fb6e24b793614ff1f6399583fbfc908c4ca21caa5e5cf0e98e983`  
- Source modified: `2026-06-21T03:19:07+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_beb40705-90b`  
- session_id: `d4b5afb5-2f03-4b58-8952-573f1381d166`
```

Transcript

1. user (2026-06-21T03:17:18.117Z)

You are ramping up an engineer on the ALREADY-WRITTEN i18n design for KhelSutra (a local AI badminton highlight indexer, Bangalore-India market).
Read VERY carefully and completely: C:/Users/avidu/Projects/badminton-highlight-indexer/docs/I18N_PLAN.md (229 lines).
Produce a thorough markdown brief capturing, with exact section/line refs where useful:

- The string-externalization model (t("key") usage, key naming convention, ICU MessageFormat plurals).
- The catalog layout (en source catalog + per-locale siblings; file format/paths the doc proposes).
- The locale-negotiation order (persisted pref -> switcher/?lang= -> Accept-Language -> en fallback).
- The translation pipeline: Gemini-DRAFT -> human-REVIEW (note the guardrail: inference only, NEVER training).
- CI enforcement: key-parity + no-bare-strings lint.
- Fonts: Noto Sans Kannada/Devanagari, the OFL licensing decision/open question.
- The phasing P0..P4 (esp. P3 = SPA react-i18next, P4 = conversational/coach-mode).
- ANY assumption the doc bakes in that is now STALE because it targeted the OLD static UI (backend/api/static/*) rather than today's surfaces (a Vite/React/TS web/ SPA + a vanilla-HTML site/). Flag each stale assumption explicitly.
- The 4 open OWNER decisions the doc raises in its section 4 (font licensing, v1 locale set, backend-localization depth, UX scope/which-surface-first) ? quote them.

Return the brief as your final message (it is data for an orchestrator, not a human chat reply).
Be exhaustive and faithful; do not invent.

2. assistant (2026-06-21T03:17:20.473Z)

I'll read the i18n plan document carefully, and also check the actual current UI surfaces to verify which assumptions are stale.

3. user (2026-06-21T03:17:21.376Z)

```
1      # Internationalization (i18n) ? Kannada-first, by design  
2  
3      **Status:** Design (P0 deliverable; phases P1+ are owner-gated, not started).  
4      **Owner decision required** before any code lands ? see [Open decisions](#open-  
decisions-for-the-owner).  
5  
6      ---  
7  
8      ## 1. Context ? why this, why now  
9  
10     The PMF beachhead is **Bangalore badminton academies**  
([`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md)).  
11     Many academy coaches, parents, and players are far more comfortable in **Kannada** than
```

English.

12 A product that only speaks English adds friction exactly at the point of first contact.
So the

13 product must speak ****Kannada first**, **Hindi next**, and extend to other Indian languages**
14 (Tamil, Telugu, Marathi, ?) ****without a refactor****.
15

16 The owner's directive is precise: make the product ****internationalizable _by design****.
Adding a

17 language must be **"add a reviewed catalog" ? not "go re-touch the code."** The translations
18 themselves can wait; ****the foundation that makes them cheap to add is the actual deliverable.****
19

20 Today there is ****zero i18n**** in the repo (no i18n/locale/gettext/babel anywhere). Every
21 user-facing string is hardcoded English across four surfaces:
22

23 | Surface | Where | Examples (today) |
24 |---|---|---|
25 | ****UI chrome**** | `backend/api/static/{index.html,app.js,style.css}` (~40?50 strings) |
`"Validate & Index Video"`, `alert("Validation Failed! ?")`, `` `\${count} Rally Segment\${count}`
> 1 ? 's' : '' } Selected` `` |
26 | ****Backend coach-facing text**** | `backend/main.py` `HTTPException(detail=?)`;
`backend/pipeline/validators/\*` `message`; `backend/reporting/spec.yaml` `customer\_verdict` +
per-rule `customer\_message` | `Video file not found at '?'`; `Video quality rejected: Focus
check failed?`; `"? Your highlights are ready."` |
27 | ****Self-navigating NL queries**** (owner ask, future) | not built yet | **"show me all
rallies > 15 shots where I won" ? the **query and its answer** must work in the user's language**
|
28 | ****Coach-mode recommendations**** (owner ask, future) | not built yet | localized
recommendation text, not English |
29

30 A React/Vite SPA is planned to replace the static UI ([`UI\_ALPHA\_EXECUTION\_PLAN.md`],
ADR-009)

31 in a ****separate private repo `avidullu/khelsutra`****. The design below is therefore
deliberately

32 ****framework-agnostic****: the catalog format and locale-negotiation contract are shared,
so the

33 static UI today and the SPA tomorrow consume the ***same*** translation source.
34

35 ****Scope confirmed with owner:**** UI + coach-facing backend text + (designed-for, built-
later)

36 NL/coach features. ****Translations are Gemini-drafted, human-reviewed**** ? this is
inference
37 use of a paid LLM, which the [licensing guardrail](#guardrail-alignment) explicitly
allows
38 (it is ***not*** a training-data source). The owner reviews the first draft of every string.
39
40 ---
41

42 **## 2. Core architecture**
43

44 **### 2.1 The "by design" invariant** ? no hardcoded user-facing strings
45

46 Every coach-facing string is a ****stable key**** resolved at render time against the active
locale.
47 Code references `t("compile.button")`, never the English prose. This is the single rule
that
48 makes "add a language = add a catalog" true, and it is ****enforced by CI**** (\$2.6) so it
cannot
49 silently regress as the codebase grows.
50
51 ---
52 "Validate & Index Video" ? t("video.action.validate_index")
53 alert("Validation Failed! ?review the checklist") ? t("toast.validation_failed")

```

54     `${count} Rally Segment${count>1?'s':''} Selected`? t("compile.selected", {count}) #
ICU plural
55     HTTPException(detail="Video file not found at ?")? detail = tr(lang,
"error.video_not_found", {path})
56     ```
57
58     ### 2.2 One canonical English source catalog, per-locale siblings
59
60     ```
61     locales/
62     en/  common.json  # source of truth ? devs author keys here
63     kn/  common.json  # Kannada (Gemini-draft ? human-review)
64     hi/  common.json  # Hindi (next; first exercise of the extensibility checklist)
65     ```
66
67     - **Format: i18next-flavored JSON** ? works for the vanilla shim today *and*
react-i18next in
68     the SPA, with no conversion.
69     - **ICU MessageFormat** for interpolation and plurals: `{count, plural, one {# rally}
other {# rallies}}`.
70     This matters immediately ? the current UI already hand-rolls English plurals
71     (`Rally Segment${count > 1 ? 's' : ''}`), and Kannada/Hindi plural rules differ from
English.
72     ICU encodes each language's rules in the catalog, not the code.
73     - `en` is the source of truth; **every other locale is keyed identically** (enforced by
the
74     key-parity check in §2.6).
75
76     ### 2.3 Locale-negotiation contract (shared frontend ? backend)
77
78     One resolution order, defined **in one place** and honored by both UI and backend:
79
80     ```
81     persisted user preference ? UI switcher / ?lang=<code> ? Accept-Language header ?
default "en"
82     ```
83
84     - Supported locales are enumerated in **a single module** (one list to grow). Anything
outside
85     it falls back to `en`.
86     - The backend localizes its own responses (`HTTPException.detail`, report `customer_`)
from the
87     same `lang` query param / `Accept-Language` header, so a Kannada UI gets Kannada
errors.
88     - **Always-fallback to `en`** for any missing key ? a missing translation must degrade
to
89     English, never to a blank or a raw key in front of a user.
90
91     ### 2.4 Backend text ? keys + params, not prose
92
93     Validator `message`s and API errors today bake in English prose *and* interpolated
values
94     (`f"?sharpness score of {avg} is below ? threshold of {threshold}."`). The design
separates
95     the two:
96
97     - The validator/endpoint emits a **stable code + a params dict**
(`("blur.below_threshold",
98     {"score": avg, "threshold": threshold})).
99     - The **prose lives in the catalog**, looked up at the API boundary in the negotiated
locale.
100    - `ValidationResult` keeps an English `message` for logs/back-compat, but gains an
optional
101    `message_key` + `message_params`; the report/serializer prefers the key when present.
This

```

```

102     keeps internal/technical messages English (good for debugging) while customer-facing
text
103     localizes.
104
105     `spec.yaml` `customer_verdict` and per-rule `customer_message` become **keyed** (or per-
locale
106     overlays keyed by rule `code`); the technical `message` blocks stay English (they cite
107     `README#Qn` and are for the operator, not the end user).
108
109     ### 2.5 Indic font strategy ? (open decision ? see §4)
110
111     The bundled UI/watermark fonts (Outfit, Roboto Slab) **do not cover Kannada or
Devanagari**.
112     **Noto Sans Kannada / Noto Sans Devanagari are OFL-licensed**, which is the same tension
we hit
113     with the watermark font: OFL is not in the strict **MIT/Apache/BSD** code/data/weights
guardrail.
114     Fonts are *media assets*, not code or model weights, and OFL is explicitly designed for
115     unrestricted redistribution ? so the cleanest path is to **carve out an explicit OFL-
for-fonts
116     exception**, the same way we'd treat any bundled media. Options put to the owner in §4.
117
118     Note the **video watermark already takes a `font_path`** (`WatermarkConfig.font_path`,
119     `backend/pipeline/stitcher/compiler.py`), so stamping a Kannada/Hindi watermark is a
120     *configuration* change (point it at a Noto font) once the font decision is made ? no new
code.
121
122     ### 2.6 CI enforcement (the guardrail that keeps it "by design")
123
124     Two checks, both pure-Python and runnable in this repo (no GPU/torch needed):
125
126     1. **Key-parity test** ? every locale catalog has *exactly* the `en` key set; fail on
missing
127     keys, extra keys, or untranslated placeholder markers (e.g. a "__MISSING__"
sentinel left
128     by the draft script). This is also the **extensibility proof**: drop a stub `ta.json`
with
129     one key and CI immediately lists every gap.
130     2. **No-bare-strings lint** ? a check that flags user-facing string literals not routed
through
131     t() / the backend tr() (scoped to the UI handlers and the API/validator/report
text
132     paths). Starts as a curated allowlist so it can land without boiling the ocean.
133
134     ### 2.7 Translation pipeline
135
136     ```
137     dev authors en keys ? scripts/i18n_draft.py (Gemini, INFERENCE only) ? kn/hi draft
138                        ? native-speaker review (owner reviews first draft)
139                        ? CI key-parity + placeholder check ? merge
140     ```
141
142     `scripts/i18n_draft.py` reads `locales/en/*.json`, asks Gemini for each missing key in
the
143     target locale **with the key name and any ICU placeholders preserved**, and writes a
draft
144     catalog with every machine-drafted value flagged for review. It never touches `en`,
never
145     invents keys, and is a *drafting aid* ? a human reviews before merge. (Reuses the
existing
146     Gemini provider credentials/config; this is the same "paid LLM = inference, never
training"
147     posture as the rest of the system.)
148
149     ---

```

```

150
151     ## 3. Phased execution (owner-gated; ONE PR per slice, off `master`)
152
153     | Phase | Scope | Repo | Gate |
154     |---|---|---|---|
155     | **P0** | **This design doc** (`docs/I18N_PLAN.md`) | this | _the immediate
deliverable_ |
156     | **P1** | Foundation + **static-UI retrofit**: `locales/` + `en` source catalog
(extract the ~40?50 UI strings ? keys), a tiny vanilla **`t(key, vars)`** shim + `data-i18n`
wiring in `index.html`/`app.js`, **language switcher (en/kn)** + `localStorage` persistence +
`?lang=`, the font decision applied, and the **`kn`** catalog (Gemini-draft ? owner review).
Reuse the existing static mount / `serve_index` in `backend/main.py`. | this | owner approval |
157     | **P2** | **Backend + report localization**: `backend/i18n/` with `tr(lang, key,
params)` (same catalog format), refactor validator `message`s to keys+params, localize
`HTTPException.detail` + `spec.yaml` `customer_verdict`/`customer_message`, wire `Accept-
Language`/?lang=. | this | owner approval |
158     | **P3** | **SPA adoption**: **react-i18next** consuming the *same* `locales/` catalogs;
documented FE?BE contract. | `avidullu/khelsutra` | needs repo access; or owner/Opus executes
there |
159     | **P4** | **Conversational + coach-mode** (future): locale-aware NL query intake ?
**templated localized answers** ; coach-mode recommendation **templates**. Ties to the owned-
model / PLATFORM $5A + rich-metadata work. | this + model | gated on those features existing |
160
161     **Extensibility checklist** (this *is* the by-design contract ? runs the same every
time):
162     > add `locales/<code>/common.json` ? `scripts/i18n_draft.py --locale <code>` ? human
review ?
163     > add `<code>` to the supported-locales list ? confirm font coverage ? CI key-parity
green.
164
165     **Hindi (`hi`)** is the first real exercise of this checklist** ? if adding `hi` requires
touching
166     anything other than the steps above, the design failed and we fix the design, not paper
over it.
167
168     ---
169
170     ## 4. Open decisions (for the owner)
171
172     1. **Target-UI scope** ? confirm the *shared framework-agnostic foundation* (prove on
the static
173     UI now, SPA adopts the same catalogs later). Alternative: scope only to the SPA and
skip the
174     static-UI retrofit. _Recommendation: shared foundation ? it's the cheap, low-risk way
to
175     validate the catalog/negotiation contract before the SPA exists._
176     2. **Indic font licensing** ? (a) **allow OFL for bundled fonts** and self-host Noto
Sans
177     Kannada/Devanagari _(recommended; treats fonts as media, not code/weights); (b) rely
on
178     **system Indic fonts** (no bundle, inconsistent rendering across machines); (c)
**Google Fonts
179     CSS** (runtime external dependency, privacy/offline concerns for a local-first
product). This
180     also unblocks the **Kannada/Hindi video watermark** (needs a Noto `font_path`).
181     3. **v1 locale set & fallback** ? confirm `en` + `kn` for v1, `hi` next; always-fallback
to `en`
182     for missing keys.
183     4. **Backend-localization depth for v1** ? recommendation: localize the **customer
verdict /
184     report `customer_`** and **validation rejection reasons** first (highest user
impact);
185     defer technical/operator messages (they stay English by design).
186
187     ---

```

```

188
189     ## 5. Key files / seams
190
191     - **Localize:** `backend/api/static/{index.html,app.js,style.css}`; `backend/main.py`
192       (`HTTPException.detail` + the `serve_index`/static mount);
193       `backend/pipeline/validators/`
194       (`message` ? key+params; see `validators/base.py::ValidationResult`);
195       `backend/reporting/spec.yaml` (`customer_verdict` + per-rule `customer_message`).
196     - **New:** `locales/{en,kn,hi}/`, `backend/i18n/` (`tr()` + supported-locales +
negotiation),
197       `scripts/i18n_draft.py`, a CI key-parity test under `tests/` + a no-bare-strings check
wired
198       into `.github/workflows/ci.yml`.
199     - **Fonts:** reuse the bundled-asset pattern in `backend/assets/fonts/` (Apache-2.0
Roboto Slab
200       ships there today); add Noto Indic fonts there *iff* decision §4.2(a).
201     - **SPA:** `avidullu/khelsutra` (react-i18next) ? separate repo.
202     ---
203
204     ## 6. Verification
205
206     - **CI key-parity test** (Python, this repo): every locale has every `en` key; fail on
207       missing/extra/placeholder. Drop a stub `ta.json` ? CI flags every gap (the
extensibility proof).
208     - **Static UI:** load `?lang=kn` ? rendered strings resolve from the `kn` catalog;
switcher
209       persists across reload; any missing key falls back to English (no raw keys shown).
210     - **Backend:** request with `Accept-Language: kn` (or `?lang=kn`) ? localized error
`detail` +
211       report `customer_`; unit tests for fallback, ICU interpolation, and plural rules.
212     - **Fonts:** Kannada + Devanagari render correctly ? **manual eyeball on the owner's
machine**
213       (the cloud container can't render fonts).
214     - **Docs-only P0 PR:** internal links resolve; indexed in [`docs/README.md`](README.md);
215       cross-linked from [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) (the NL/coach
surfaces)
216       and [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (the Bangalore beachhead).
217
218     ---
219
220     ## Guardrail alignment
221
222     - **Paid LLM = inference only.** Gemini drafts translations (inference); it is **never**
a
223       training-data source. ? consistent with [`CLAUDE.md`](../CLAUDE.md) and the ratified
guardrail.
224     - **Licensing.** Code stays MIT/Apache-2.0/BSD. The *only* tension is **fonts** (OFL) ?
handled
225       as an explicit, owner-approved media-asset exception in §4.2, not a silent dependency.
226     - **No minors / biometrics scope creep.** i18n touches presentation only; it introduces
no new
227       personal-data collection.
228
229     [`UI_ALPHA_EXECUTION_PLAN.md`]: ./UI_ALPHA_EXECUTION_PLAN.md
230

```

4. user (2026-06-21T03:17:25.320Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/doc-status-register' from the remote, but no such ref was fetched.

---STATUS---

backend/api/static

STATIC EXISTS

NO web/

NO site/

5. assistant (2026-06-21T03:17:28.361Z)

Now let me verify the actual current state of the UI surfaces to ground the stale-assumption analysis. Let me check the static UI, the khelsutra SPA, and look for any `web/` or `site/` style surfaces.

6. user (2026-06-21T03:17:31.906Z)

```
=== static UI dir ===
total 48
drwxr-xr-x 1 avidu 197609    0 Jun 11 20:44 .
drwxr-xr-x 1 avidu 197609    0 Jun 19 13:03 ..
-rw-r--r-- 1 avidu 197609 16350 Jun 11 20:44 app.js
-rw-r--r-- 1 avidu 197609  8743 Jun  7 14:14 index.html
-rw-r--r-- 1 avidu 197609 13418 Jun  7 14:14 style.css
=== fonts dir ===
LICENSE-Apache-2.0.txt
README.md
RobotoSlab-Regular.ttf
=== locales? ===
NO locales/ yet
=== scripts i18n? ===
NO i18n_draft.py yet
=== backend i18n? ===
NO backend/i18n yet
```

7. user (2026-06-21T03:17:34.295Z)

```
=== khelsutra root ===
total 38
drwxr-xr-x 1 avidu 197609    0 Jun 20 23:27 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 06:51 ..
drwxr-xr-x 1 avidu 197609    0 Jun 20 15:20 .claude
drwxr-xr-x 1 avidu 197609    0 Jun 21 08:47 .git
-rw-r--r-- 1 avidu 197609  297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609   78 Jun 20 18:40 .gitignore
-rw-r--r-- 1 avidu 197609 5823 Jun 20 23:06 NEXT_STEPS.md
-rw-r--r-- 1 avidu 197609  925 Jun 20 08:37 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 20 23:14 docs
drwxr-xr-x 1 avidu 197609    0 Jun 21 08:20 site
drwxr-xr-x 1 avidu 197609    0 Jun 20 22:08 web
=== web/ ? ===
web
README.md
dist
index.html
node_modules
package-lock.json
package.json
src
tsconfig.json
vite.config.ts
=== site/ ? ===
site
README.md
access.db
make_og.py
node_modules
package-lock.json
package.json
public
```

```
schema.sql
scripts
server.js
=== vite/package ===
=== git remote ===
origin https://github.com/avidullu/khelsutra.git (fetch)
origin https://github.com/avidullu/khelsutra.git (push)
```

8. assistant (2026-06-21T03:17:38.285Z)

The khelsutra repo confirms today's surfaces: a `web/` Vite/React/TS SPA and a separate `site/` (Node `server.js` + `public/`). Let me inspect both to ground the stale-assumption flags precisely.

9. user (2026-06-21T03:17:40.940Z)

```
=== web/package.json ===
{
  "name": "khelsutra-web",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer engine).",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit",
    "test": "vitest run"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "@tailwindcss/vite": "^4.0.0",
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "@vitejs/plugin-react": "^4.3.4",
    "tailwindcss": "^4.0.0",
    "typescript": "^5.7.0",
    "vite": "^6.0.0",
    "vitest": "^3.0.0"
  }
}
=== web/src tree ===
web/src:
App.tsx
api
components
hooks
index.css
lib
main.tsx

web/src/api:
client.test.ts
client.ts
types.ts

web/src/components:
CompileResult.tsx
QueryBar.examples.test.ts
QueryBar.tsx
RallyGrid.tsx
```

SelectionFooter.tsx

web/src/hooks:
useRallySelection.ts

web/src/lib:
floor.test.ts
floor.ts
format.test.ts
format.ts
log.ts
query.test.ts
query.ts
selection.test.ts
selection.ts
=== web has i18next? ===
no i18next in web/package.json

10. user (2026-06-21T03:17:42.567Z)

```
=== site/package.json ===
{
  "name": "khelsutra-site",
  "private": true,
  "type": "module",
  "scripts": {
    "test": "node --test",
    "test:cf": "node --test src/index.test.js",
    "test:vm": "node --test src/server.test.js",
    "test:deploy": "node --test src/deploy-config.test.js src/assets.test.js",
    "deploy": "wrangler deploy",
    "dev": "wrangler dev",
    "og": "python make_og.py"
  },
  "devDependencies": {
    "jsdom": "^29.1.1"
  }
}
=== site/public ===
capture.html
img
index.html
og.png
own-model.html
video
=== site type ===
// Always-on VM host for the KhelSutra site ? dependency-free (Node 22+ only).
// Serves public/ and POST /api/request-access into SQLite (node:sqlite), using the
// SAME handler logic as the Cloudflare Worker (src/index.js). This is the portability
// fallback "if Cloudflare acts out" ? see docs/PORTABILITY.md.
//
// PORT=8080 DB_PATH=/var/lib/khelsutra/access.db node server.js
//
import { createServer } from "node:http";
import { readFile } from "node:fs/promises";
import { join, extname, normalize } from "node:path";
import { DatabaseSync } from "node:sqlite";
import { handleRequest } from "./src/index.js";
import { sqliteStore } from "./src/store.js";
import { noopNotify } from "./src/notify.js";

const PORT = Number(process.env.PORT) || 8080;
const ROOT = import.meta.dirname;
const PUBLIC = join(ROOT, "public");
const DB_PATH = process.env.DB_PATH || join(ROOT, "access.db");
```

```
const db = new DatabaseSync(DB_PATH);
db.exec(await readFile(join(ROOT, "schema.sql"), "utf8")); // CREATE TABLE IF NOT EXISTS ?
idempotent
const store = sqlLiteStore(db);
const notify = noopNotify(); // VM host: no email yet (add SMTP later ? see docs/PORTABILITY.md)
```

```
const TYPES = {
  ".html": "text/html; charset=utf-8", ".png": "image/png", ".svg": "image/svg+xml",
  ".css": "text/css", ".js": "text/javascript", ".ico": "image/x-icon", ".txt": "text/plain",
  ".mp4": "video/mp4", ".webm": "video/webm", ".gif": "image/gif", ".jpg": "image/jpeg",
  ".jpeg": "image/jpeg",
};
=== khelsutra README ===
```

KhelSutra Guru ?

****Private product repo**** ? the heart and brain of khelsutra.guru:
the web app (alpha SPA) now, the hosted control plane at beta.

- ****Engine**** (pipeline, API, eval): sibling repo `badminton-highlight-indexer` (rename pending).
- ****Canonical planning docs**** (live in the engine repo's `docs/` for now, migrate here at beta):
`UI\_PROPOSAL.md` · `HOSTING\_PLAN.md` · `UI\_ALPHA\_EXECUTION\_PLAN.md`.
- ****Status:**** pre-alpha scaffold. SPA lands in `web/` per execution-plan milestone M1.

Structure

- `site/` ? public marketing landing page (static; served at khelsutra.guru)
- `web/` ? React SPA (alpha; Vite + React + TS + Tailwind; Cloudflare Pages build root)
- `docs/` ? product docs as they migrate from the engine repo
- `control-plane/` ? (beta) Cloud Run FastAPI control plane

License

All rights reserved (private; no OSS license).

11. assistant (2026-06-21T03:19:07.715Z)

I now have everything needed to ground the analysis. The doc's central assumption ? that the SPA lives in a "planned" separate repo and that the static `backend/api/static/\*` UI is "today's" surface ? is confirmed stale: the `avidullu/khelsutra` repo already exists with a live `web/` Vite/React/TS SPA and a separate `site/` vanilla-HTML marketing landing page. Here is the brief.

i18n Design Ramp-Up Brief ? KhelSutra (`docs/I18N\_PLAN.md`, 229 lines)

****Doc status (line 3):**** "Design (P0 deliverable; phases P1+ are owner-gated, not started)."
Owner sign-off required before any code lands (line 4). Tagline: ****"Internationalization (i18n) ? Kannada-first, by design" (line 1).**

****One-line thesis (lines 16?18):**** make the product internationalizable **by design** so that adding a language is **"add a reviewed catalog"*, never **"go re-touch the code."* The translations can wait ? ****the foundation that makes them cheap to add is the actual P0 deliverable.****

****Market driver (lines 10?14):**** PMF beachhead = Bangalore badminton academies; coaches/parents/players often more comfortable in ****Kannada****. Order: ****Kannada first, Hindi next****, then Tamil/Telugu/Marathi "without a refactor."

1. String-externalization model (§2.1, lines 44?56)

- **The invariant:** **no hardcoded user-facing strings.** Every coach-facing string is a **stable key** resolved at render time against the active locale. Code references ``t("compile.button")``, **never the English prose** (lines 46?47). This single rule is what makes "add a language = add a catalog" true, and it is **CI-enforced** (§2.6) so it cannot silently regress.
- ``t("key")`` frontend / ``tr(lang, key, params)`` backend are the two accessors.
- **Key naming convention** ? dotted, hierarchical, namespaced-by-surface/intent. Exact examples the doc commits to (lines 52?56):
 - ``"Validate & Index Video"`` ? ``t("video.action.validate_index")``
 - ``alert("Validation Failed! ?")`` ? ``t("toast.validation_failed")``
 - ```${count} Rally Segment${count>1?'s':''} Selected`` ? ``t("compile.selected", {count})`` **(ICU plural)**
 - ``HttpException(detail="Video file not found at ?")`` ? ``detail = tr(lang, "error.video_not_found", {path})``
- **ICU MessageFormat for interpolation + plurals** (§2.2, lines 69?72): ``"{count, plural, one {# rally} other {# rallies}}"``. Rationale called out explicitly: the current UI hand-rolls English plurals (``Rally Segment${count > 1 ? 's' : ''}``), and **Kannada/Hindi plural rules differ from English** ? ICU encodes each language's rules **in the catalog, not the code.**

2. Catalog layout (§2.2, lines 58?74)

```

...
locales/
  en/  common.json  # source of truth ? devs author keys here
  kn/  common.json  # Kannada  (Gemini-draft ? human-review)
  hi/  common.json  # Hindi    (next; first exercise of the extensibility checklist)
...

- Format: i18next-flavored JSON (line 67) ? deliberately chosen to work for both the vanilla shim today *and* react-i18next in the SPA, *with no conversion*.
- en is the single source of truth; every other locale is keyed identically to en, enforced by the key-parity check (§2.6) (lines 73?74).
- Proposed file/seam paths (§5, lines 195?199): new dirs `locales/{en,kn,hi}/`, `backend/i18n/`, `scripts/i18n_draft.py`; fonts under `backend/assets/fonts/`.

```

3. Locale-negotiation order (§2.3, lines 76?89)

One resolution order, defined in one place, honored by both UI and backend (line 81):

```

...
persisted user preference ? UI switcher / ?lang=<code> ? Accept-Language header ? default "en"
...

```

- Supported locales enumerated in **a single module** (one list to grow); anything outside it falls back to **en** (lines 84?85).
- Backend localizes its **own** responses (``HttpException.detail``, report ``customer_``) from the **same** ``lang`` query param / ``Accept-Language`` header, so a Kannada UI gets Kannada errors (lines 86?87).
- **Always-fallback to en** for any missing key ? a missing translation degrades to English, never to a blank or a raw key shown to a user (lines 88?89).

4. Backend text ? keys + params (§2.4, lines 91?107)

- Today validators/errors bake English prose **and** interpolated values together (``f"?sharpness score of {avg} is below ? threshold of {threshold}."``). Design **separates the two**:
 - Validator/endpoint emits a **stable code + params dict** ? e.g. ``("blur.below_threshold", {"score": avg, "threshold": threshold})`` (lines 97?98).
 - **Prose lives in the catalog**, looked up at the API boundary in the negotiated locale (line 99).
 - ``ValidationResult`` keeps an **English message** for logs/back-compat but **gains optional message_key + message_params**; the report/serializer prefers the key when present. Internal/technical messages stay English (debugging) while customer-facing text localizes (lines 100?103).
- ``spec.yaml`` ``customer_verdict`` + per-rule ``customer_message`` become **keyed** (or per-locale

overlays keyed by rule `code`); the technical `message` blocks stay English (they cite `README#Qn`, are for the operator) (lines 105?107).

5. Translation pipeline ? Gemini-DRAFT ? human-REVIEW (§2.7, lines 134?147; also §1 lines 35?38)

...

```
dev authors en keys ? scripts/i18n_draft.py (Gemini, INFERENCE only) ? kn/hi draft
      ? native-speaker review (owner reviews first draft)
      ? CI key-parity + placeholder check ? merge
```

...

- `scripts/i18n\_draft.py` reads `locales/en/\*.json`, asks Gemini for each **missing** key in the target locale **with the key name and any ICU placeholders preserved**, and writes a draft catalog with **every machine-drafted value flagged for review** (lines 142?144).
- It **never touches `en`**, never invents keys, and is a **drafting aid only** ? a human reviews before merge (lines 144?145).
- **GUARDRAIL** (load-bearing, stated twice ? lines 37?38, 146?147, and `$"Guardrail alignment"` lines 222?223): **Gemini is used for inference only** (drafting); it is **NEVER** a training-data source. This is the same "paid LLM = inference, never training" posture as the rest of the system, consistent with `CLAUDE.md` and the ratified licensing guardrail. The owner reviews the first draft of **every** string.

6. CI enforcement (§2.6, lines 122?132) ? "the guardrail that keeps it by design"

Two checks, **both pure-Python, runnable in this repo (no GPU/torch)** (line 124):

1. **Key-parity test** (lines 126?129) ? every locale catalog has **exactly** the `en` key set; **fail on missing keys, extra keys, or untranslated placeholder markers** (e.g. a `"__MISSING__"` sentinel left by the draft script). Doubles as the **extensibility proof**: drop a stub `ta.json` with one key ? CI immediately lists every gap.
2. **No-bare-strings lint** (lines 130?132) ? flags user-facing string literals **not routed** through `t()` / backend `tr()` (scoped to UI handlers + API/validator/report text paths). **Starts as a curated allowlist** so it can land without "boiling the ocean."

Wired into `.github/workflows/ci.yml`; tests under `tests/` (§5, lines 196?197). Verification restates the parity test + the stub-`ta.json` extensibility proof (§6, lines 206?207).

7. Fonts ? Indic font strategy (§2.5, lines 109?120; decision in §4.2)

- Bundled UI/watermark fonts (**Outfit, Roboto Slab**) **do not cover Kannada or Devanagari** (line 111).
- **Noto Sans Kannada / Noto Sans Devanagari are OFL-licensed** (line 112) ? same tension as the watermark font: **OFL is not in the strict MIT/Apache/BSD** code/data/weights guardrail.
- **Recommended resolution:** fonts are **media assets**, not code or model weights; OFL is designed for unrestricted redistribution ? **carve out an explicit OFL-for-fonts exception**, treated like any bundled media (lines 114?116).
- **Watermark is already config-only** (lines 118?120): the video watermark already takes a `font_path` (`WatermarkConfig.font_path`, `backend/pipeline/stitcher/compiler.py`), so a Kannada/Hindi watermark is just pointing it at a Noto font once the decision is made ? **no new code**.
- Today's bundled-font asset pattern lives at `backend/assets/fonts/` (currently Apache-2.0 Roboto Slab); add Noto Indic there **iff** decision §4.2(a) (§5, lines 198?199). **(Verified on disk: `backend/assets/fonts/` contains only `RobotoSlab-Regular.ttf` + `LICENSE-Apache-2.0.txt` ? no Noto fonts yet.)**

8. Phasing P0?P4 (§3, lines 151?166)

Phase	Scope	Repo	Gate
P0	This design doc	(<code>docs/I18N_PLAN.md</code>)	this the immediate deliverable
P1	Foundation + static-UI retrofit	: <code>locales/</code> + <code>en</code> source catalog (extract ~40?50 UI strings ? keys); a tiny vanilla <code>t(key, vars)</code> shim + <code>data-i18n</code> wiring in <code>index.html/app.js</code> ; language switcher (en/kn) + <code>localStorage</code> persistence + <code>?lang=</code> ;	

font decision applied; the `kn` catalog (Gemini-draft ? owner review). Reuse existing static mount / `serve_index` in `backend/main.py`. | this | owner approval |

| **P2** | **Backend + report localization**: `backend/i18n/` with `tr(lang, key, params)` (same catalog format); refactor validator `message`'s ? keys+params; localize `HTTPException.detail` + `spec.yaml` `customer_verdict`/`customer_message`; wire `Accept-Language`/`lang`. | this | owner approval |

| **P3** | **SPA adoption: react-i18next** consuming the *same* `locales/` catalogs; documented FE?BE contract. | `avidullu/khelsutra` | needs repo access; or owner/Opus executes there |

| **P4** | **Conversational + coach-mode** (future): locale-aware NL query intake ? **templated localized answers**; coach-mode recommendation **templates**. Ties to owned-model / PLATFORM \$5A + rich-metadata work. | this + model | gated on those features existing |

Extensibility checklist (lines 161?163) ? *this is the by-design contract*:

> add `locales/common.json` ? `scripts/i18n_draft.py --locale <code>` ? human review ? add `<code>` to supported-locales list ? confirm font coverage ? CI key-parity green.

Hindi (hi) is the first real exercise (lines 165?166): if adding `hi` requires touching anything beyond those steps, "the design failed and we fix the design, not paper over it."

9. STALE ASSUMPTIONS ? doc targets the OLD static UI, not today's surfaces

The doc was written when the only concrete UI was `backend/api/static/` and the SPA was merely *planned* in a "separate private repo." *That repo now exists and is materially further along than the doc assumes.* Verified on disk in `C:\Users\avidu\Projects\khelsutra`: a live `web/` Vite + React 19 + TS + Tailwind 4 SPA (`web/package.json`, `web/src/App.tsx`, components, vitest) *and* a separate `site/` vanilla-HTML marketing landing page (Node `server.js` + `public/{index.html,capture.html,own-model.html,...}`, Cloudflare Worker `src/index.js`). Each stale assumption, flagged:

1. *"Today there is a static UI at `backend/api/static/`; the SPA is *planned* / *tomorrow*"* ? STALE. Lines 20?33 frame `backend/api/static/{index.html,app.js,style.css}` as "the static UI today" and the SPA as "planned to replace the static UI ? in a separate private repo `avidullu/khelsutra`." Reality: the SPA already lives and runs in `khelsutra/web/`. The static UI is legacy/parallel, not "the present surface." Any P1 plan to *retrofit* `backend/api/static/` *first* may be retrofitting a surface that is already being superseded.

2. *The "four surfaces" table omits the `site/` vanilla-HTML marketing page entirely* ? STALE/INCOMPLETE. \$1 table (lines 23?28) enumerates UI chrome, backend coach-facing text, NL queries, coach-mode. It has *no row* for the public landing page. Today `khelsutra/site/public/` (`index.html`, `capture.html`, `own-model.html`, request-access form copy) is a real, user-facing, currently-shipping English surface ? and arguably the *first* thing a Bangalore academy sees. Its localization is unscoped by this doc.

3. *P1's "static-UI retrofit" target is misdirected* ? STALE. Line 156 spends P1 on extracting the ~40?50 strings from `backend/api/static/` + a "tiny vanilla `t(key,vars)` shim" + `data-i18n` wiring in `index.html/app.js`, reusing `serve_index` in `backend/main.py`. With the SPA already built in React, the *vanilla-shim retrofit* of the legacy static UI may be throwaway work. The two genuinely-current frontend surfaces are (a) the React `web/` SPA (which needs react-i18next, i.e. doc's **P3**, not P1) and (b) the `site/` HTML (which needs *some* shim, but it's a different codebase/repo than the one P1 names).

4. *Phase ordering assumes static-UI (P1) before SPA (P3)* ? STALE / likely inverted. The doc gates P3 behind "needs repo access; or owner/Opus executes there" (line 158) as if the SPA is a future, access-blocked concern. Since `web/` exists now and has *no i18next* in its deps (verified: `web/package.json` lists react/vite/tailwind/vitest, *no* `i18next/react-i18next`), the real, current frontend i18n work is **P3-shaped and immediate**, not P1. The doc's "prove on the static UI first, SPA adopts later" sequencing (\$4.1) predates the SPA existing.

5. *"Reuse the existing static mount / `serve_index` in `backend/main.py`"* ? STALE coupling assumption. Line 156. The current SPA is a Vite build (Cloudflare Pages build root per khelsutra README) and the landing page is served by Cloudflare Worker / Node `server.js` ? *not* FastAPI's `serve_index`. The frontend?backend serving topology the doc assumes (FastAPI

serves the HTML) no longer matches how `web/` and `site/` are actually served.

6. **§5 "Key files / seams"** and §1 examples point only at `backend/api/static/\*` ? STALE inventory. Lines 191-200 list `backend/api/static/{index.html,app.js,style.css}` as *the* frontend localize-targets. The real frontend string inventory now lives in `khelsutra/web/src/components/\*` (`QueryBar.tsx`, `RallyGrid.tsx`, `CompileResult.tsx`, `SelectionFooter.tsx`, `App.tsx`) and `khelsutra/site/public/\*.html`. The string-extraction estimate ("~40-50 UI strings") is measured against the obsolete static UI, not the SPA + landing page.

7. **Repo-of-record framing** is dated ? note. Lines 30-33/200 call `avidullu/khelsutra` a "separate private repo" the SPA is "planned to" move to. It already *is* the SPA's home and is checked out locally as a sibling (`C:\Users\avidu\Projects\khelsutra`, remote `github.com/avidullu/khelsutra`). The "or owner/Opus executes there" hedge (line 158) is no longer access-blocked in practice.

> **What is NOT stale** (still valid, and the doc's actual durable value): the framework-agnostic *catalog format* (i18next JSON), the *locale-negotiation contract*, ICU plurals, the Gemini-draft?human-review pipeline with the inference-only guardrail, the CI key-parity + no-bare-strings checks, the backend keys+params refactor (§2.4 ? `backend/` is the same repo and unchanged), and the font decision. These are explicitly designed to be shared across "the static UI today and the SPA tomorrow" (lines 31-33), so they survive the surface change. The breakage is purely in **which frontend surface P1 targets and the P1-before-P3 ordering**, because the SPA arrived ahead of the doc.

10. The 4 open OWNER decisions (§4, lines 170-185) ? quoted verbatim

> **1. Target-UI scope** ? confirm the *shared framework-agnostic foundation* (prove on the static UI now, SPA adopts the same catalogs later). Alternative: scope only to the SPA and skip the static-UI retrofit. *Recommendation*: shared foundation ? it's the cheap, low-risk way to validate the catalog/negotiation contract before the SPA exists.

> **2. Indic font licensing** ? (a) **allow OFL for bundled fonts** and self-host Noto Sans Kannada/Devanagari (recommended; treats fonts as media, not code/weights); (b) rely on **system Indic fonts** (no bundle, inconsistent rendering across machines); (c) **Google Fonts CSS** (runtime external dependency, privacy/offline concerns for a local-first product). This also unblocks the **Kannada/Hindi video watermark** (needs a Noto `font\_path`).

> **3. v1 locale set & fallback** ? confirm `en` + `kn` for v1, `hi` next; always-fallback to `en` for missing keys.

> **4. Backend-localization depth for v1** ? recommendation: localize the *customer verdict* / report `customer\_\*\*\*` and *validation rejection reasons* first (highest user impact); defer technical/operator messages (they stay English by design).

Note for the owner-decision discussion: Decision #1 is the one most directly invalidated by §9 above ? its premise ("prove on the static UI now ? before the SPA exists") is counterfactual: the SPA exists. The decision should be re-framed as *which of the two live surfaces* (`web/` SPA via react-i18next, `site/` landing via a shim) leads*, and whether the legacy `backend/api/static/\*` retrofit is worth doing at all.

Source references

- Primary doc (read in full): `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\I18N\_PLAN.md` (229 lines).
- Stale-assumption grounding (verified on disk): `C:\Users\avidu\Projects\khelsutra\web\` (Vite+React 19+TS+Tailwind SPA, `web/package.json`, `web/src/components/\*`; **no i18next dependency**), `C:\Users\avidu\Projects\khelsutra\site\` (vanilla-HTML landing, `site/server.js` + `site/public/\*.html`), `C:\Users\avidu\Projects\khelsutra\README.md`.
- Legacy static UI still present: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\static\{index.html,app.js,style.css}`.

- Fonts seam: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\assets\fonts\` (only `RobotoSlab-Regular.ttf` + Apache-2.0 license today; ****no Noto Indic fonts yet****).
- Not-yet-created scaffolding the doc proposes: `locales/`, `backend/i18n/`, `scripts/i18n\_draft.py` ? ****none exist yet**** (P1+ not started, consistent with the doc's "owner-gated, not started" status).